

AWS Bedrock 과 Strands 를 활용한 OTel 기반 Agentic AI Observability (with **New Relic**)



나요안나

Senior Technical Success Manager,
New Relic

01

발표자 소개

02

Observability란? (w/ OpenTelemetry)

03

AI의 패러다임 변화

04

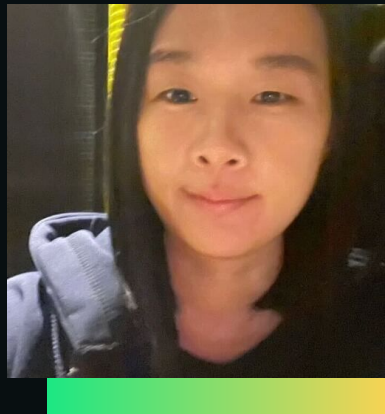
Agentic AI 애플리케이션 및 모니터링

05

Quiz

프로필

- Senior Technical Success Manager at New Relic (현)
- Customer Solutions Manager at AWS (전)
- 분야 : Cloud Architecture, APM/Infra/AIOps Observability



나요안나
Senior Technical Success Manager
New Relic



Monitoring vs Observability

	Monitoring (모니터링)	Observability (옵저버빌리티)
대상	 Monolith 앱 + 서버	 MSA (마이크로서비스)  Dynamic 인프라  Async Event-driven 패턴
목적	시스템의 징후를 실시간으로 추적하여 장애 발생을 빠르게 감지	어디가 어떻게 왜 문제가 생겼는지 미지의 장애 원인을 즉시 파악
답변 가능한 질문	Known Questions ✓ 서비스가 켜져 있는가? (Availability) ✓ 정상적으로 작동하는가? (Health) ✓ 성능은 양호한가? (Performance)	Unknown Questions ✓ 시스템을 설계시 예상 못했던 에러는 왜 발생했는가?

Telemetry 데이터 종류 (MELTx)

Metrics - 무엇이 일어났는가? (What)

시간에 따른 수치 측정값

예) CPU 사용률, 응답시간 (duration), 요청 (requestCount)

Events - 언제 일어났는가?
(When)

특정 시점에 발생한 활동

예) TransactionError, ComputeSample

Logs - 왜 일어났는가? (Why)

시스템 /애플리케이션의 텍스트 기반
상세 기록

예) 시스템/애플리케이션 로그, CloudWatch Log

Traces - 어디서 일어났는가? (Where)

시스템의 전체 경로 추적

예) 메인 agent → 하위 agent → 연결된 AI 모델 / 톨 /
API 호출 → 응답 결과값 반환

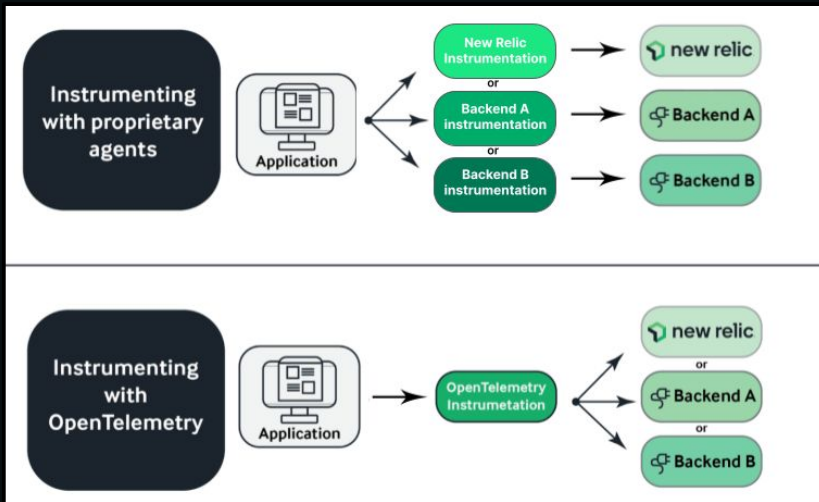
X(eXtended Data) - 어떻게 최적화?
(How)

특정 사용 사례에 최적화된 데이터

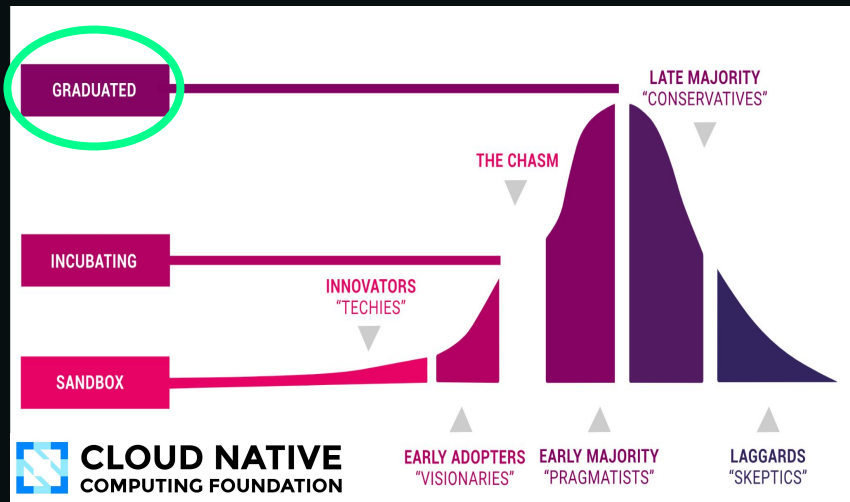
예) 비용 및 보안 관련 CustomEvent, 비즈니스
메트릭, Entity, Relationship

AI 생태계의 MCP? 011y 생태계의 OpenTelemetry

OTel (OpenTelemetry)? 애플리케이션, 인프라 등에서 다양한 Telemetry 데이터를 수집/관리하는 Toolkit

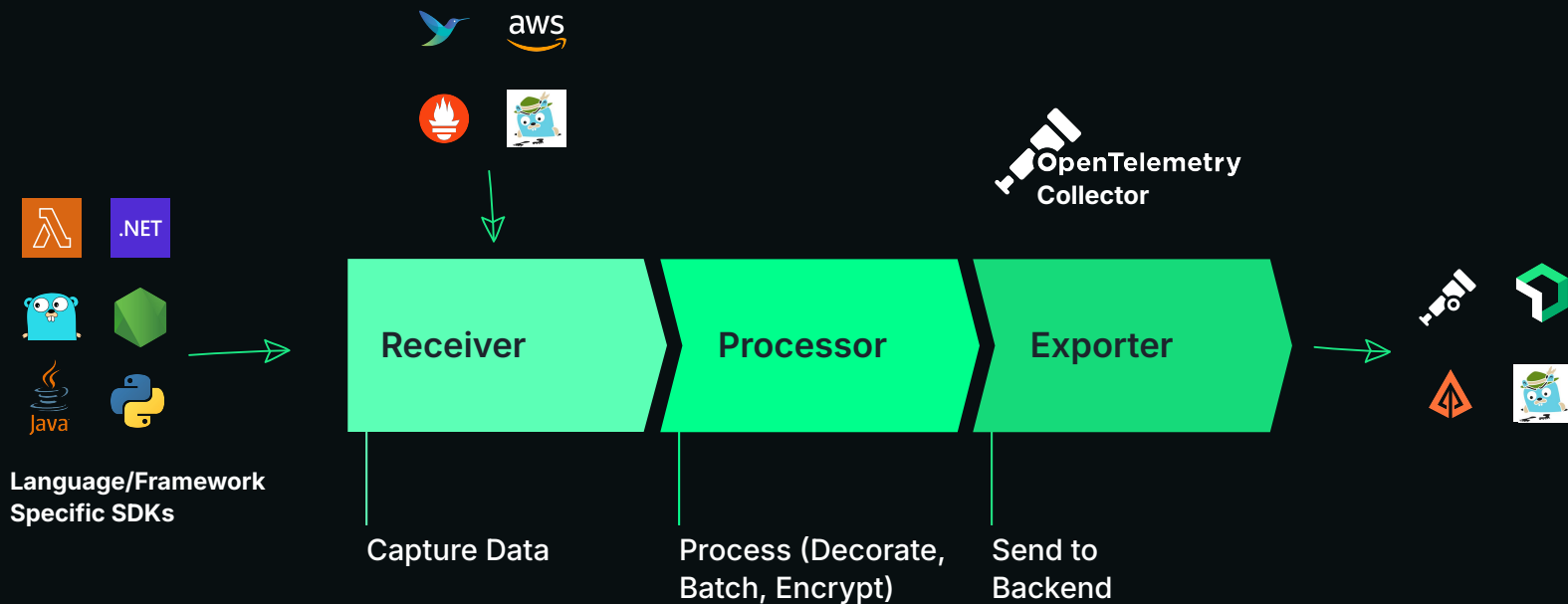


OTel을 쓰면 뭐가 좋은건가요?



CNCF 공식 오픈소스 프로젝트

Snapshot of OpenTelemetry



최근 AI 트렌드 및 키워드 변화



AI 중심으로 DevOps/AIOps 재편

82%

매주 작성하는 코드의
절반 이상이 AI에
의해 생성되거나
리팩토링되고 있음

88%

바이브코딩을 Prod
정책에 공식
반영하였고, Dev
환경으로 제한하는
경우는 5%,
완전히 금지하는
경우는 없음

94%

코드 리뷰시 AI가
생성한 코드의 품질이
사람이 작성한
코드보다 더 높음

25%

AI를 통한 Prod
배포에 대한 제어 및
관리 가시성 부재

출처: New Relic 2026 State of AI Coding Report (기업 내 테크 리더 200명 대상)

AI Monitoring 왜 필요한건가요?

AI 애플리케이션의 응답 시간/품질,비용을 모니터링하고 관리할 새로운 운영 역량 필요성

AI 테크 스택 복잡성 증가



AI 모델, 벡터 데이터스토어,
오케스트레이션 프레임워크 등의
새로운 구성 요소

새로운 Telemetry 데이터 등장



Bias, Hallucination, Toxicity 등 AI
거버넌스 및 응답 품질 측정을 위한
새로운 데이터 지표 관리

보안/컴플라이언스



급변하는 글로벌 규제 가운데
데이터 외부 유출/오용에 대한
예방

Deterministic Observability for Non-deterministic AI?



Amazon Strands

자율형 AI 에이전트 개발 및 배포 목적의
Python/Typescript 오픈소스 SDK

멀티 에이전트의 모델 기반 개발

에이전트 추론 논리 흐름 시각화/구조화

모델 메타데이터 및 비용 지표 파싱

Observability 최적화



Amazon Bedrock

단일 통합 API를 통해 FM을 완전 관리형 제공하여
GenAI 애플리케이션을 가장 쉽게 만들고
스케일링할 수 있는 서비스

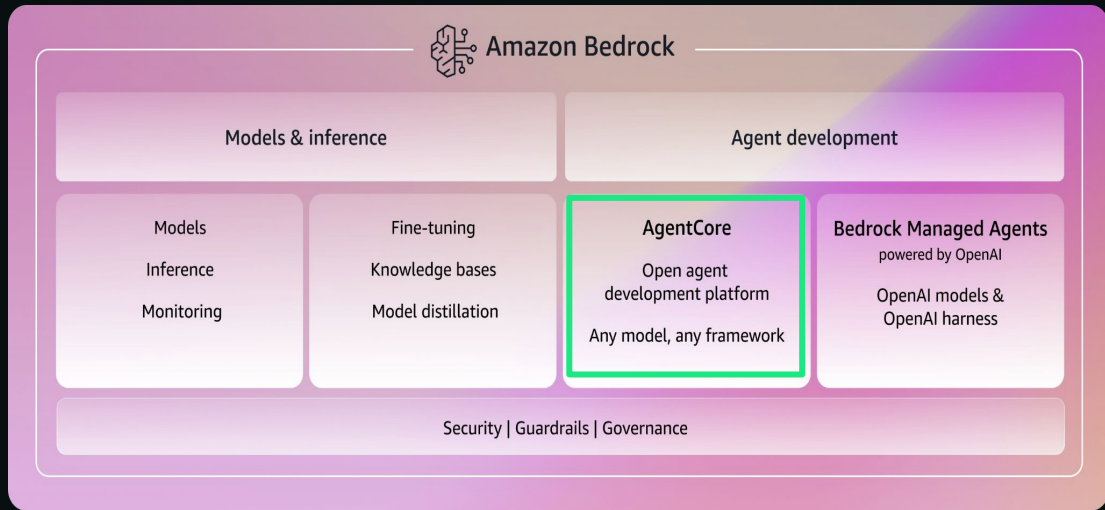
AI 모델에 대한 폭넓은 선택지

단일 통합 API를 통한 Scaling

RAG 및 자율형 에이전트 구축 간소화

데이터 보안 및 Privacy

Amazon Bedrock AgentCore



동적 도구 호출 및 실행 자동화

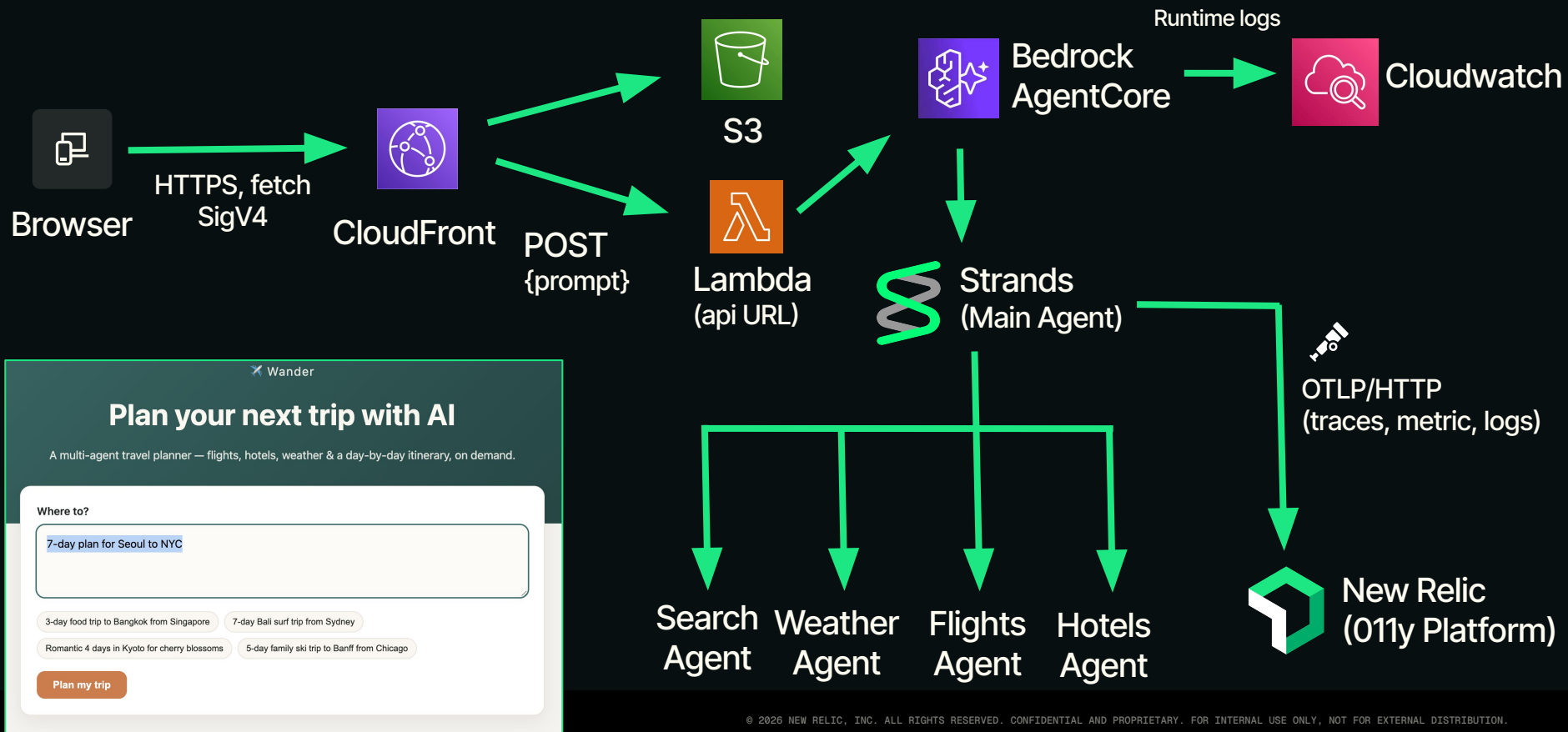
표준화된 프레임워크 기반 개발
(Pydantic, LiteLLM, FastAPI)

이벤트 기반/서버리스 연동 및 확장

가드레일 및 보안 통제 세분화/단일화

Observability 최적화 (OTel 지원)

멀티 에이전트 웹 애플리케이션 (여행 플래너) 아키텍처



멀티 에이전트 웹 애플리케이션 (여행 플래너) 아키텍처

```
agent/      AgentCore agent – Strands multi-agent app
  app.py      BedrockAgentCoreApp entrypoint (POST /invocations, GET /ping)
  main.py     local CLI runner
  agents/     orchestrator + 4 specialists + model factory
  tools/      search / weather / flights / hotels @tool functions
  observability/ OpenTelemetry setup → OTLP
  requirements.txt agent runtime deps
  Dockerfile  reference only (CDK uses direct code deploy, no container)
api/         Lambda proxy (public Function URL) → InvokeAgentRuntime
  lambda_function.py, requirements.txt
frontend/    Vanilla webapp (index.html, app.js, styles.css); reads /config.json
infra/       Python CDK app
  app.py      wires TravelAgent → TravelApi → TravelFrontend
  build.py    Docker-free packaging (agent arm64 deps, api boto3)
  stacks/     agent_stack.py · api_stack.py · frontend_stack.py
  cdk.json, requirements.txt
```

Telemetry 데이터 설정

```
# --- Traces + metrics via Strands' built-in OTel integration -----
try:
    from strands.telemetry import StrandsTelemetry

    strands_telemetry = StrandsTelemetry()
    strands_telemetry.setup_otlp_exporter()           # traces
    strands_telemetry.setup_meter(enable_otlp_exporter=True) # metrics

    from opentelemetry import trace, metrics

    _tracer_provider = trace.get_tracer_provider()
    _meter_provider = metrics.get_meter_provider()
    logger.info("Strands OTel trace + metric exporters configured.")
```

```
def _setup_log_bridge(service_name: str, endpoint: str, license_key: str):
    """Route stdlib logging records to OTLP logs (New Relic Logs)."""
    from opentelemetry._logs import set_logger_provider
    from opentelemetry.sdk._logs import LoggerProvider, LoggingHandler
    from opentelemetry.sdk._logs.export import BatchLogRecordProcessor
    from opentelemetry.exporter.otlp.proto.http._log_exporter import OTLPLogExporter
    from opentelemetry.sdk.resources import Resource

    resource = Resource.create({"service.name": service_name})
    provider = LoggerProvider(resource=resource)
    set_logger_provider(provider)

    exporter = OTLPLogExporter(
        endpoint=f"{endpoint.rstrip('/')}/v1/logs",
        headers={"api-key": license_key},
    )
    provider.add_log_record_processor(BatchLogRecordProcessor(exporter))

    # Attach to the root logger so all app logs are forwarded.
    handler = LoggingHandler(level=logging.INFO, logger_provider=provider)
    logging.getLogger().addHandler(handler)
    return provider
```

AWS Native Log 데이터 설정

```
# Load .env first (no-op in the container if absent), then telemetry, then
# import the agent so the model client is built after OTel is configured.
load_dotenv()
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)

from observability import setup_telemetry # noqa: E402

setup_telemetry("travel-agent-agentcore")

from bedrock_agentcore.runtime import BedrockAgentCoreApp # noqa: E402

from agents.orchestrator import run_travel_query # noqa: E402

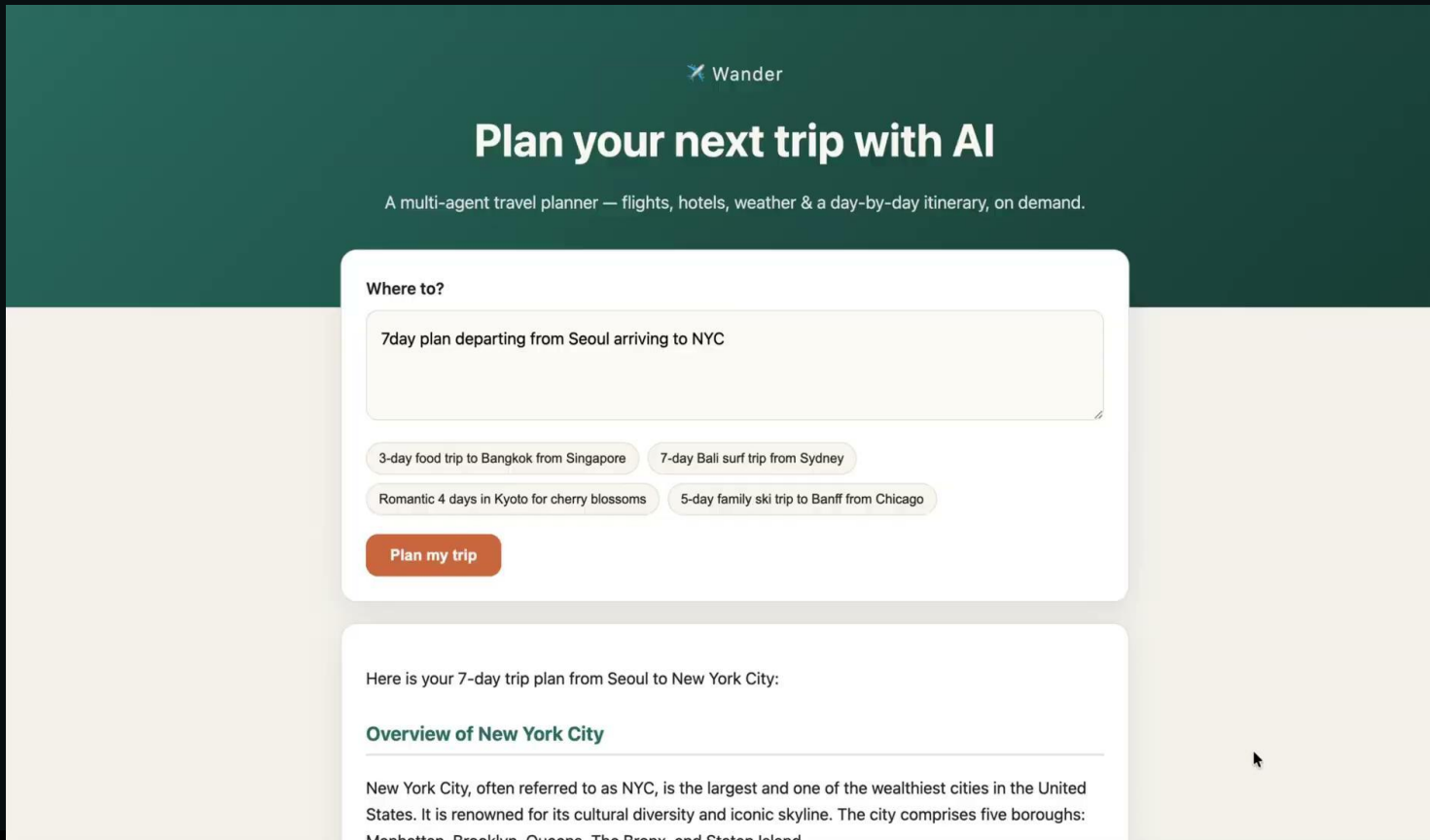
logger = logging.getLogger(__name__)
app = BedrockAgentCoreApp()
```

에이전트 생성 및 연결

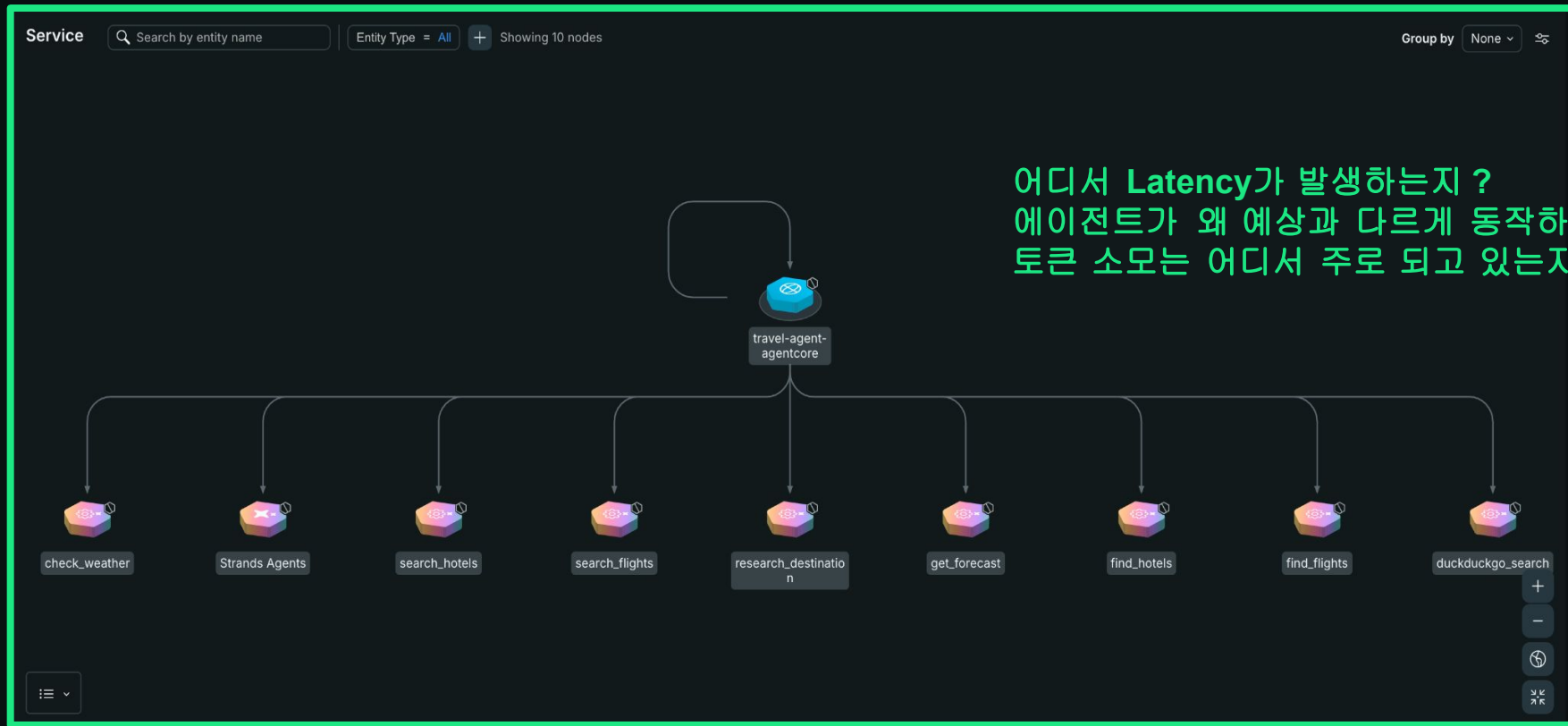
```
def build_orchestrator(trace_attributes: dict | None = None) -> Agent:
    """Build the orchestrator agent.

    Args:
        trace_attributes: Optional span attributes (e.g. session/user IDs) used
            to correlate traces for this invocation in the observability
            backend.
    """
    return Agent(
        model=get_model(),
        system_prompt=ORCHESTRATOR_SYSTEM_PROMPT,
        tools=[research_destination, check_weather, find_flights, find_hotels],
        trace_attributes=trace_attributes or {},
    )
```

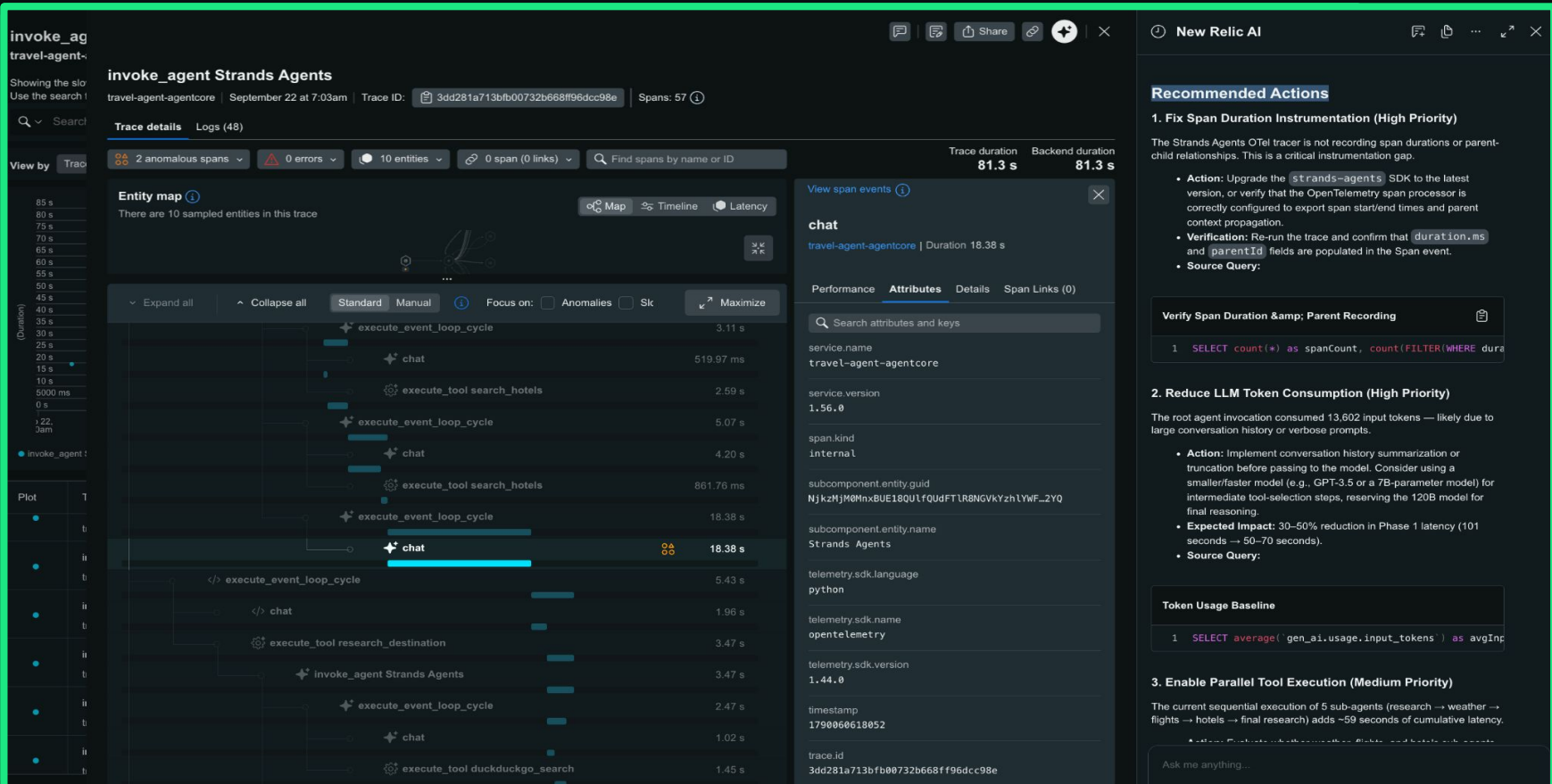
모델별 응답 출력 결과 비교



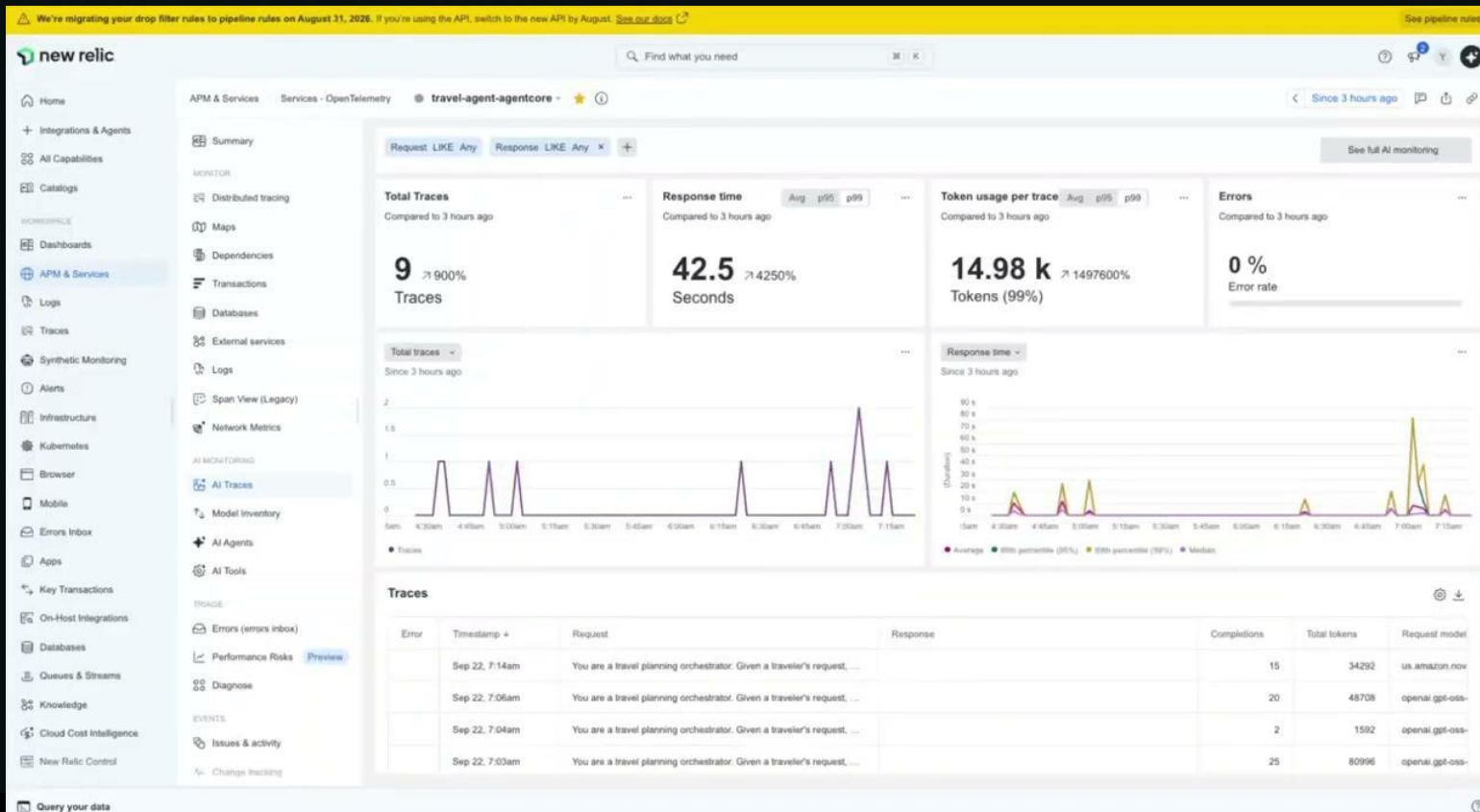
AI 에이전트 모니터링의 어려움



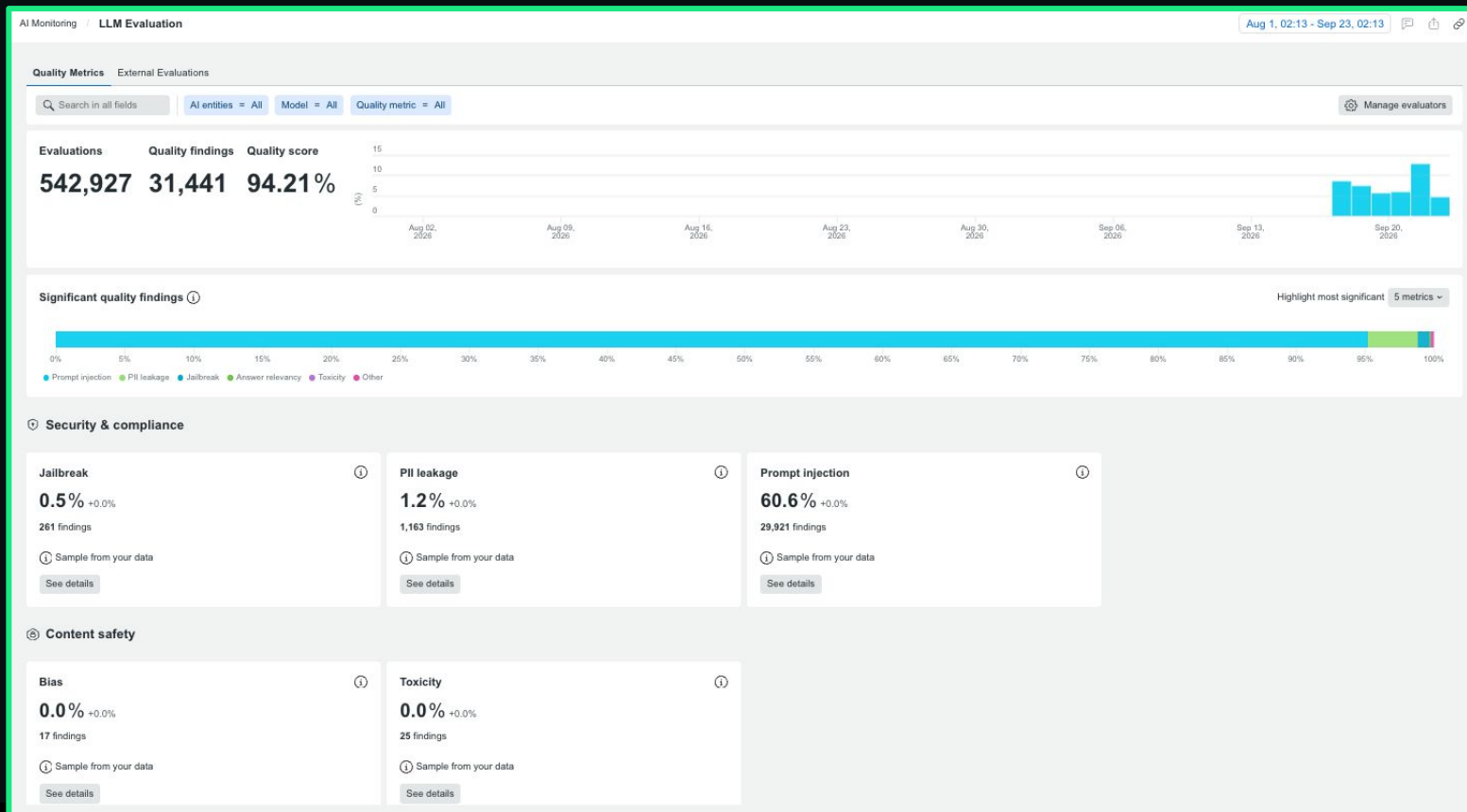
Distributed Tracing



New Relic AI Monitoring



AI Evaluation/Governance



Quiz



Quiz 1

오피저버빌리티 생태계에서 특정 벤더 종속 없이
애플리케이션 및 인프라의 관측 데이터를 수집하고
관리하는 CNCF 공식 오픈소스 표준 툴킷은 무엇인가요 ?

1. OpenTelemetry (OTel)
2. Model Context Protocol (MCP)
3. Nagios
4. CloudWatch

Quiz 1

오픈서빌리티 생태계에서 특정 벤더 종속 없이
애플리케이션 및 인프라의 관측 데이터를 수집하고
관리하는 CNCF 공식 오픈소스 표준 툴킷은 무엇인가요?

1. **OpenTelemetry (OTel)**
2. **Model Context Protocol (MCP)**
3. **Nagios**
4. **CloudWatch**

Quiz 2

다음 중 Amazon Bedrock AgentCore와 Amazon Strands SDK의 아키텍처 및 역할에 대한 설명으로 가장 적절한 것은 무엇인가요?

1. Strands SDK는 가상 환경의 오케스트레이션 프레임워크로, Bedrock AgentCore와는 독립적으로 동작하여 상호 연동이 불가능하다.
2. AgentCore는 Strands 에이전트를 감싸는 경량 실행 환경 역할을 하며, Strands SDK는 모델 기반의 추론 경로 시각화 및 자동 토큰 및 비용 파싱 기능을 제공한다.
3. Strands SDK는 AWS Lambda 함수 자체를 대체하는 서버리스 인프라 프로비저닝 도구이다.
4. AgentCore는 외부 API 호출을 전담하며, 내부 비즈니스 로직과 에이전트 생성은 Strands가 담당한다.

Quiz 2

다음 중 Amazon Bedrock AgentCore와 Amazon Strands SDK의 아키텍처 및 역할에 대한 설명으로 가장 적절한 것은 무엇인가요?

1. Strands SDK는 가상 환경의 오케스트레이션 프레임워크로, Bedrock AgentCore와는 독립적으로 동작하여 상호 연동이 불가능하다.
2. AgentCore는 Strands 에이전트를 감싸는 경량 실행 환경 역할을 하며, Strands SDK는 모델 기반의 추론 경로 시각화 및 자동 토큰 및 비용 파싱 기능을 제공한다.
3. Strands SDK는 AWS Lambda 함수 자체를 대체하는 서버리스 인프라 프로비저닝 도구이다.
4. AgentCore는 외부 API 호출을 전담하며, 내부 비즈니스 로직과 에이전트 생성은 Strands가 담당한다.

Quiz 3

다음 중 New Relic AI Monitoring (AIM)이 제공하는 주요 기능이 아닌 것은 무엇인가요 ?

1. 모델별 토큰 사용량 및 비용 실시간 추적
2. 정확도 (Accuracy), 환각 (Hallucination), 독성 (Toxicity) 등의 응답 품질 지표 관리
3. 프론트엔드단 브라우저부터 AWS Lambda와 AgentCore 런타임까지 아우르는 분산 추적 (Distributed Tracing)
4. 자율형 파운데이션 모델 직접 학습 및 파인튜닝 서비스

Quiz 3

다음 중 New Relic AI Monitoring (AIM)이 제공하는 주요 기능이 아닌 것은 무엇인가요?

1. 모델별 토큰 사용량 및 비용 실시간 추적
2. 정확도 (Accuracy), 환각 (Hallucination), 독성 (Toxicity) 등의 응답 품질 지표 관리
3. 프론트엔드단 브라우저부터 AWS Lambda와 AgentCore 런타임까지 아우르는 분산 추적 (Distributed Tracing)
4. 자율형 파운데이션 모델 직접 학습 및 파인튜닝 서비스



(온라인 웨비나 소개) New Relic NOW 2026

End-to-end 옴저버빌리티를 통한 AI-driven 운영 방식

The poster features the New Relic logo at the top left, followed by the word 'NOW' in large, glowing green letters and '2026' in white. Below this, the text 'The future of AI-driven operations' is displayed in white. At the bottom, event details are listed: 'AMER: Tuesday, October 6, 2026' with a calendar icon, '9AM PT / 12PM ET' with a clock icon, 'APAC / EMEA: Wednesday, October 7, 2026' with a calendar icon, '2:30PM AEDT / 9AM IST | 11:30AM SGT & 9AM GMT' with a clock icon, and 'Free Live Virtual Event' with a location pin icon. The background is dark blue with abstract geometric shapes and a yellow vertical bar on the right.

 new relic

NOW 2026

The future of AI-driven operations

 **AMER:** Tuesday, October 6, 2026

 9AM PT / 12PM ET

 **APAC / EMEA:** Wednesday, October 7, 2026

 2:30PM AEDT / 9AM IST | 11:30AM SGT & 9AM GMT

 Free Live Virtual Event

[웨비나 등록 링크](#)

Main Agenda

1

Scale trusted AI actions - Build the foundation of operational data to safely deploy AI at scale

2

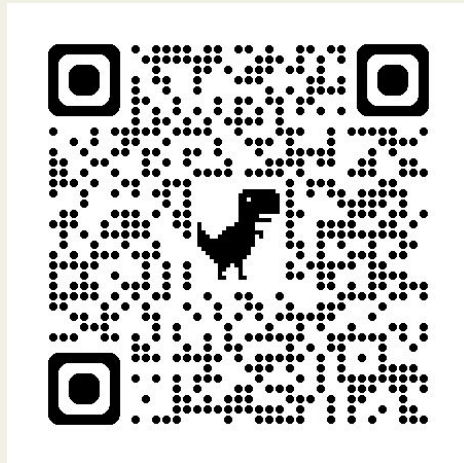
Shrink tool spend - Cut software licensing costs and eliminate complex integration maintenance

3

Lower MTTR drastically - Reduce incident response times from 45+ minutes to 3 minutes using governed and automated AI resolution



(자료 공유) 샘플 에이전트 앱 **GitHub Repository**



[Github 링크](#)



Q&A





감사합니다!



NEWRELIC.COM